Advanced Data Structures
Project

Job Recommendation System

Developed by

1. *Vijayakumar R (2024115111)*
2. *Haran S (2024115123)*
3. *Shriya L (2024115105)*

Mentor : Dr. Abirami Murugappan

Institution : College of Engineering , Guindy

INDEX

INTRODUCTION

The Job Recommendation System is a web-based application designed to efficiently manage and retrieve job-related information using advanced data structures. It enables users to explore job listings, perform keyword-based searches using job titles or skills, view recently accessed jobs, and filter jobs based on salary.

The system utilizes data structures such as **Trie, Splay Tree, and Leftist Heap** to optimize searching, tracking, and filtering operations. Job data, including job ID, title, skills, and salary, is processed efficiently to ensure fast response times and improved user experience, even for large datasets.

PROJECT DESCRIPTION

The Job Recommendation System is a browser-based application that enables efficient management and retrieval of job-related information using advanced data structures . The system begins with a homepage where all available job listings are displayed.

When a user performs actions such as searching or applying for a job, **the input is captured by the React frontend and passed through JavaScript to C++ modules. These modules implement advanced data structures and are executed in the browser using WebAssembly, enabling high-performance processing.**

The system supports functionalities such as keyword-based job search, tracking recently viewed jobs, and salary-based filtering. Each feature is powered by a specific data structure to ensure efficiency and scalability. The processed results are returned and dynamically rendered in the user interface.

This design ensures a smooth and efficient flow of data within the browser, resulting in a responsive job recommendation system.

DATA STRUCTURES USED :

TRIES	Used for efficient keyword-based job search using prefix matching.
SPLAY TREE	Used to maintain recently viewed jobs with the most recent job at the top.
LEFTIST TREE	• Used for salary-based job filtering and efficient merging of job datasets.

FEATURES :

Job Search using Trie

1. The user enters a job title or skill in the search bar on the frontend.
2. The input is captured by React and sent to the backend via JavaScript.
3. The keyword is processed by the Trie implemented in C++.
4. The Trie performs prefix-based search to find matching jobs.
5. The results are returned via WebAssembly and displayed in the UI

Recently Viewed Jobs using Splay Tree

1. When a user clicks the "Apply" button, the job ID is captured.
2. The job ID is sent from the frontend to the backend.
3. The backend inserts the job ID into the Splay Tree.
4. The inserted node is splayed to the root to mark it as recent.
5. The updated list is returned via WebAssembly and shown with the latest job on top.

Job Filtering using Leftist Heap

1. The user selects a filter option based on salary criteria.
2. The request is sent from the frontend to the backend.
3. Jobs are inserted into separate Leftist Heaps based on the condition.
4. The heaps are merged using an efficient merge operation.
5. The final sorted jobs are returned via WebAssembly and displayed in the UI.

CHOICE OF DATA STRUCTURES :

Trie

- The Trie is used for keyword-based job search with a **time complexity of O(L)**, where L is the length of the input string.
- It enables fast prefix-based searching, which is ideal for matching job titles and skills.
- It avoids scanning the entire dataset, **unlike arrays or lists which take O(n) time.**
- It is chosen over linear structures because it significantly reduces search time for large datasets.

Splay Tree

- The Splay Tree supports operations with an **amortized time complexity of O(log n).**
- It moves the most recently accessed element to the root using splaying.
- This improves access time for frequently used jobs without extra storage.
- It is preferred over arrays or linked lists, **which require shifting or reordering with O(n) cost.**

Leftist Heap

- The Leftist Heap supports **merge operations in O(log n) time.**
- It efficiently combines two heaps, making it suitable for merging filtered job datasets.
- It maintains heap order to allow quick retrieval of highest salary jobs.
- It is chosen over binary heaps, where **merging takes O(n) time, making it less efficient.**

TECH STACK :

React: Used to build a dynamic and responsive user interface for handling user interactions and rendering job data efficiently.

C++: Used to implement advanced data structures with high performance and efficient memory management.

WebAssembly (WASM): Used to execute C++ code in the browser and enable seamless integration between JavaScript and backend logic with near-native speed.

CONCLUSION :

The Job Recommendation System demonstrates the effective use of advanced data structures to improve searching, tracking, and filtering operations.

By integrating Trie, Splay Tree, and Leftist Heap with WebAssembly, the system achieves efficient performance and a responsive user experience. Overall, it highlights the importance of choosing suitable data structures for building optimized applications.